

Developer Guide: Abroad Hub

1. Introduction

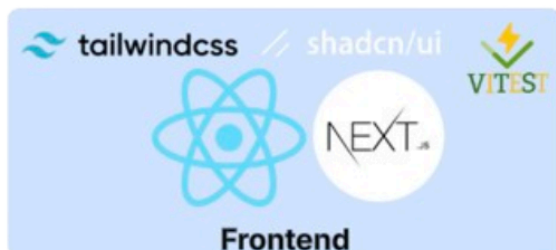
Welcome to the **Abroad Hub** developer guide! This document provides an overview of the system architecture, technologies used, setup instructions, and key development workflows.

2. System Overview

2.1 High-Level Architecture

- **Backend:** Django with Django REST Framework (DRF)
- **Frontend:** Next.js with React and shadcn/ui
- **Database:** PostgreSQL
- **Authentication:** JWT Authentication
- **API Communication:** RESTful API with JSON responses
- **Hosting:** PM2, Gunicorn, Nginx
- **CI/CD:** GitHub Actions

2.2 Technology Stack



3. Setting Up the Development Environment

3.1 Prerequisites

Ensure you have the following installed:

- **Python 3.10+**
- **Node.js (LTS version)**
- **PostgreSQL**
- **Git**

3.2 Backend Setup (Django)

1. Clone the repository
`git clone https://github.com/amanda-menza/abroad-hub.git`
`cd abroad-hub`
2. Create and activate a virtual environment
`python -m venv venv`
`source venv/bin/activate` # On Windows: `venv\Scripts\activate`
3. Install dependencies
`pip install -r requirements.txt`
4. Set up environment variables
Create a `.env` file in the project root:
`DB_HOST = ""`
`DB_PORT= "5432"`
`DB_USER = ""`
`DB_NAME=""`
`DB_PWD = ""`
5. Apply database migrations
`python manage.py makemigrations`
`python manage.py migrate`
6. Create a superuser (for admin access)
`python manage.py createsuperuser`
7. Run the development server
`python manage.py runserver`

3.3 Frontend Setup (Next.js)

1. Navigate to the frontend directory
`cd frontend`
2. Install dependencies
`npm install`
3. Set up environment variables
Create a `.env.local` file in the `frontend/` directory:
`NEXT_PUBLIC_API_URL=http://Your-Url`
4. Run the Next.js development server
`npm run dev`
5. Open `http://localhost:3000` in your browser.

4. Database Schema

4.1 Table Overview

AppUser Extends the User model and adds additional attributes:

- **display_name**: A customizable name for display purposes.
- **roles**: Defines the role of the user (Student, Administrator, Reviewer, Faculty) with a default of 'Student'.
- **dob**: Date of birth, validated to ensure it's at least 10 years ago.
- **is_sso**: A boolean indicating whether the user uses Single Sign-On.
- **profile**: A one-to-one relationship with the UserProfile model.
- **use_mfa**: Boolean field for enabling Multi-Factor Authentication (MFA).
- **requires_otp()**: Method to determine if the user needs OTP verification.

UserProfile Stores additional user-related attributes:

- **major**: User's major (optional).
- **gpa**: User's GPA, validated between 0.0 and 4.0.
- **str()**: Returns a string representation of the profile, displaying the associated user's display name.

Program Defines a study abroad program:

- **title**: The name of the program.
- **year**: The academic year of the program.
- **semester**: The semester (Fall, Spring, or Summer).
- **location**: The program's location.

- **faculty_leads**: Many-to-Many relationship with AppUser for faculty leads.
- **description**: A text description of the program.
- **start_date, end_date, open_date, deadline, essential_doc_deadline**: Various important dates for the program.
- **update_completed_applications()**: Updates application statuses based on program end date.
- **clean()**: Ensures date constraints (e.g., start date before end date).

Application Represents a student's application to a program:

- **student**: A ForeignKey linking to an AppUser.
- **program**: A ForeignKey linking to a Program.
- **status**: Application status (e.g., Applied, Eligible, Approved, Enrolled, Completed, Canceled, Withdrawn).
- **submission_date**: Timestamp of when the application was submitted.
- **essential documents**: File fields for required documents (e.g., risk form, housing questionnaire, medical records).
- **clean()**: Ensures logical constraints on status transitions.

Question Defines a question related to a program:

- **program**: ForeignKey to a Program.
- **text**: The text of the question.

Answer Stores responses to program questions:

- **application**: ForeignKey to an Application.
- **question**: ForeignKey to a Question.
- **response**: The user's answer.

ConfidentialNote Stores confidential notes for applications:

- **application**: ForeignKey to an Application.
- **author**: ForeignKey to an AppUser.
- **content**: The text of the note.
- **timestamp**: Timestamp of note creation.

Content Stores static content for the application:

- **content**: Default text describing study abroad opportunities.
- **save()**: Ensures only one instance of Content exists.

DocumentTemplate Defines document templates for required application documents:

- **document_type**: Type of document (e.g., Risk Form, Code of Conduct, Housing Questionnaire, Medical Records).
- **template_file**: The file template.

- **updated_at**: Timestamp of last update.

RecommendationLetter Manages recommendation letter requests:

- **application**: ForeignKey to an Application.
- **writer_name, writer_email**: Details of the recommender.
- **status**: Status of the recommendation request (Pending, Fulfilled).
- **token**: Unique identifier for the request.
- **letter_file**: The uploaded recommendation letter.
- **delete()**: Ensures physical file deletion upon record deletion.

4.2 Relationships Summary:

- User ↔ AppUser (OneToOne)
- AppUser ↔ UserProfile (OneToOne, nullable)
- AppUser ↔ Application (OneToMany, as "student")
- AppUser ↔ Program (ManyToMany, as "faculty_leads")
- Program ↔ Application (OneToMany)
- Application ↔ ConfidentialNote (OneToMany, as "notes")
- Application ↔ RecommendationLetter (OneToMany, as "recommendation_letters")
- Application contains multiple essential document uploads with timestamps.
- Program ↔ Question (OneToMany, as "questions")
- Application ↔ Answer (OneToMany, as "answers")

5. API Summary

Applications API

- **GET /api/applications/**: Lists all applications.
- **POST /api/applications/create/<int:program_id>/**: Creates a new application for a given program.
- **GET /api/applications/<int:pk>/**: Retrieves a specific application.
- **PUT/PATCH /api/applications/<int:pk>/**: Updates an application.
- **DELETE /api/applications/<int:pk>/**: Deletes an application.
- **PATCH /api/applications/<int:pk>/change_status/**: Changes the status of an application.
- **POST /api/applications/<int:pk>/upload-document/**: Uploads a document to an application.
- **GET /api/check-application/<int:program_id>/**: Checks if a user has already applied to a

program.

User API

- **GET/POST/PUT/PATCH /api/user/profile/**: Retrieves, creates, or updates user profiles.
- **GET /api/current-user/**: Retrieves the currently authenticated user's details.
- **GET /api/user/<int:pk>/**: Retrieves details of a specific user.

Programs API

- **GET/POST /api/programs/**: Lists all programs or creates a new program.
- **POST /api/programs/create/**: Creates a new program.
- **GET /api/programs-student-list/**: Lists programs available for students.
- **GET/PUT/PATCH/DELETE /api/programs/<int:pk>/**: Retrieves, updates, or deletes a specific program.
- **PUT /api/programs/<int:pk>/edit/**: Updates a program.
- **GET /api/program-application-counts/**: Retrieves the number of applications per program.
- **GET /api/myprograms/**: Retrieves programs the current student has applied to.
- **GET /api/programs/<int:program_id>/admin/**: Retrieves administrative details for a specific program.

Authentication & User Management

- **POST /api/change-password/**: Allows a user to change their password.
- **POST /api/reset-password/<int:user_id>/**: Resets a student's password.
- **GET /api/user-application-counts/**: Retrieves the number of applications submitted by a user.
- **GET /api/user-program-counts/**: Retrieves the number of programs a user is associated with.
- **GET /api/user-management/**: Lists all users for management.
- **PATCH/DELETE /api/user-management/<int:pk>/**: Updates a user's role or deletes the user.

Faculty API

- **GET /api/faculty-lead-program-counts/<int:pk>/**: Retrieves the number of programs led by a faculty member.

- **GET /api/faculty-user-query/**: Searches for faculty users.

SSO Authentication

- **GET /api/sso-auth-redirect/**: Redirects users to SSO authentication.
- **GET /api/sso-token-redirect/<str:code>/**: Handles token exchange after SSO authentication.

Application Documents & Notes

- **GET /api/applications/<int:application_id>/documents/**: Retrieves documents related to an application.
- **GET/POST /api/applications/<int:application_id>/notes/**: Lists or adds notes to an application.
- **GET/PUT/PATCH/DELETE /api/applications/<int:application_id>/notes/<int:pk>/**: Manages a specific note on an application.

Document Management

- **GET /api/document-templates/**: Lists available document templates.
- **GET /api/secure-document/<int:application_id>/<str:document_type>/**: Retrieves a secure document.
- **GET /api/secure-recommendation-letter/<int:letter_id>/**: Retrieves a secure recommendation letter.

Recommendation Letters

- **GET/POST /api/applications/<int:application_id>/recommendation-letters/**: Lists or creates recommendation letters for an application.
- **GET/PUT/DELETE /api/recommendation-letters/<int:pk>/**: Manages a specific recommendation letter.
- **POST /api/recommendation-letters/token/<uuid:token>/**: Submits a recommendation letter via a public token-based endpoint.

Miscellaneous

- **POST /api/update-dob/**: Updates the date of birth for a user.
- **GET /api/content/**: Retrieves application-related content.